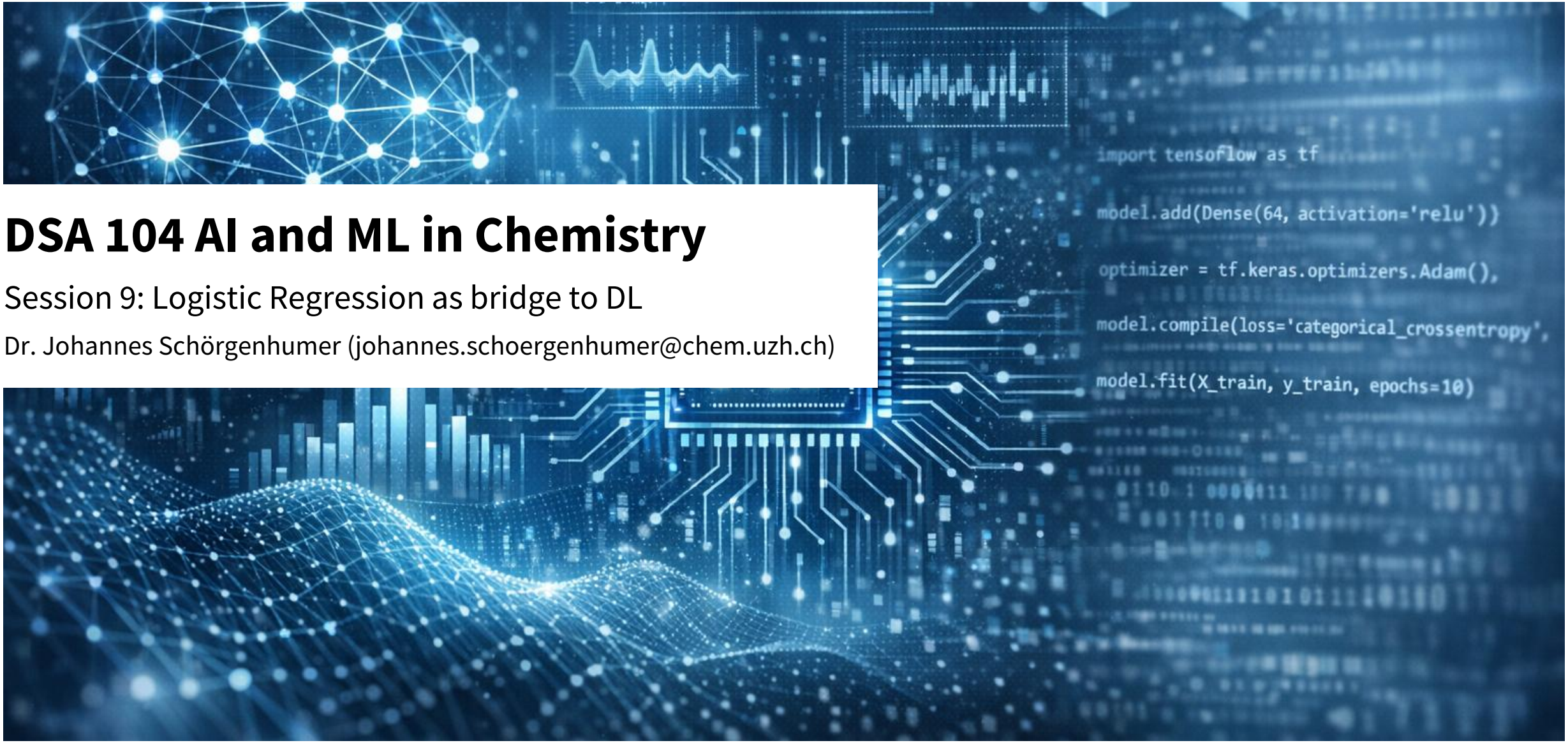
Universität Zürich[UZH]

Department of Chemistry



DSA 104 AI and ML in Chemistry

Session 9: Logistic Regression as bridge to DL

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

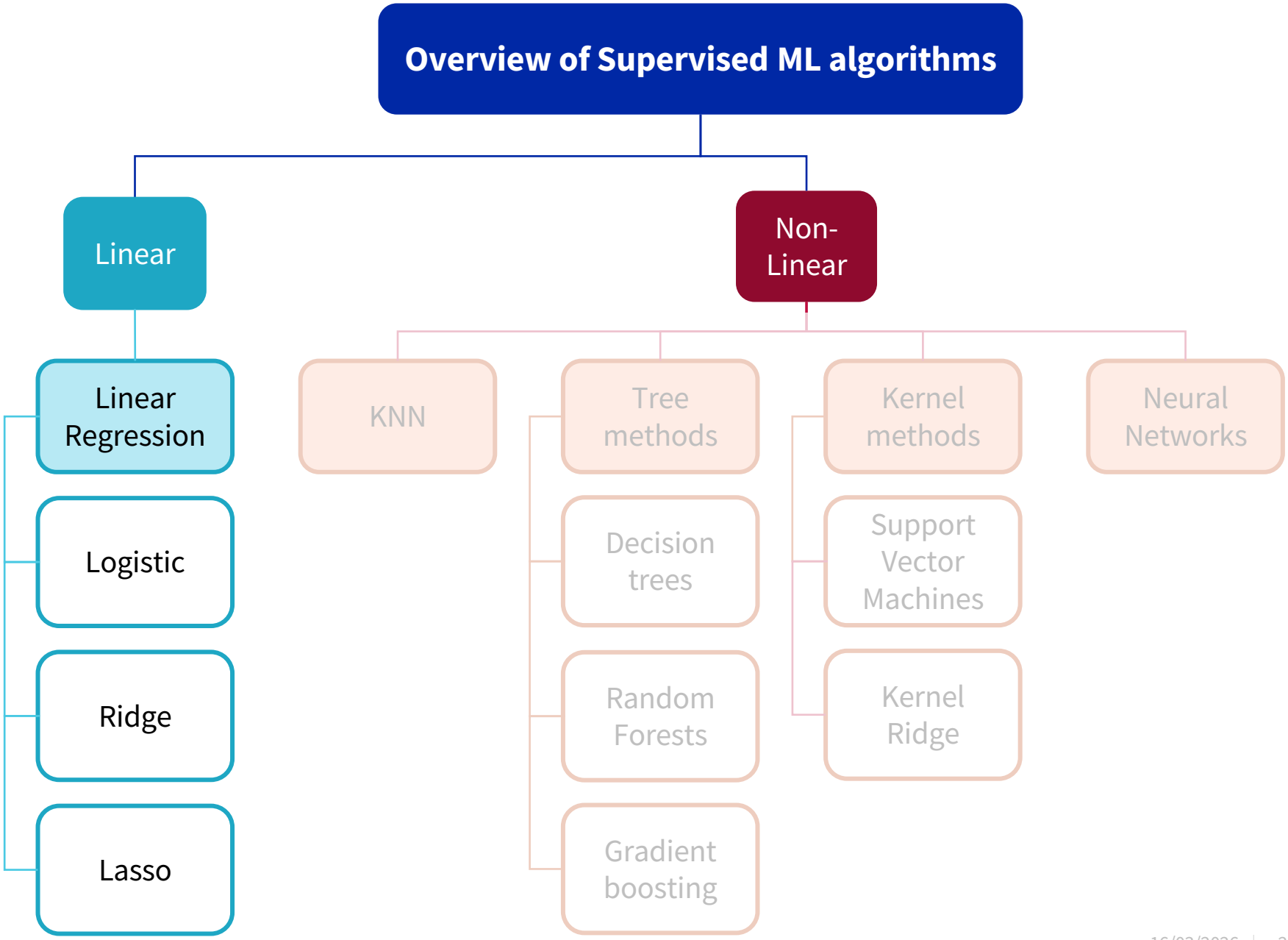
Lesson outline

Linear regression

Logistic regression

Towards DL

PyTorch



Why linear models?

- **Prediction problems**
  - House prices
  - Energy demand
  - Sensor values

- **Classification problems**
  - Spam detection
  - Disease diagnosis
  - Fraud detection

One of the simplest ML algorithms!

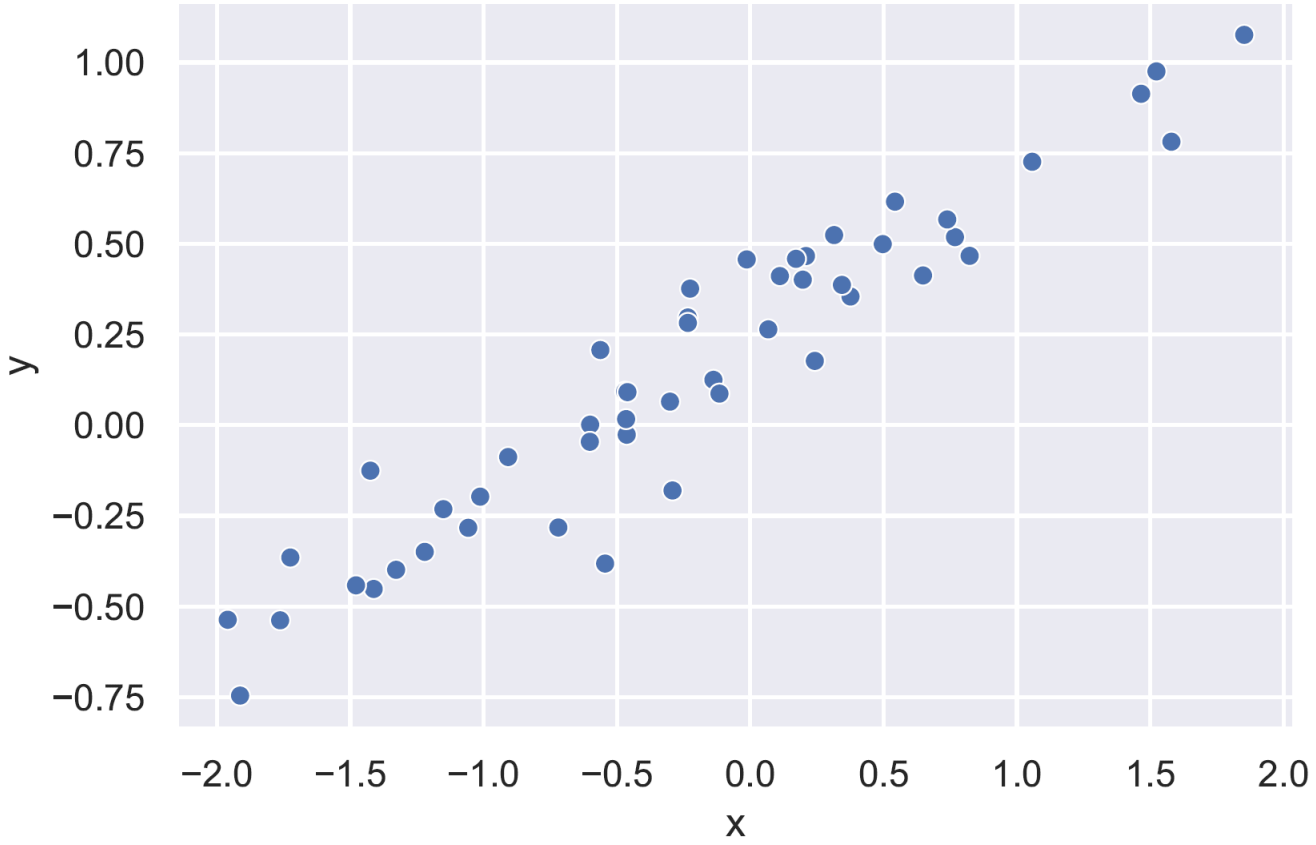
Linear regression → Logistic regression → Neural Networks → Deep Learning

- **Linear models are important because they are:**
  - **interpretable**
  - mathematically **tractable**
  - computationally **efficient**

Outset

- Labelled data: $(X, y) = ((x_1, y_1), \dots, (x_n, y_n))$
  - x_i : feature vector
  - y_i : corresponding label
  - n : Number of samples

- Find model $g(x; w)$ so that $\forall i$: $g(x_i; w) \approx y_i$
  - …where for Regression: $y_i \in \mathbb{R}$
  - …and for Classification: $y_i \in \{1, 2, \dots, K\}$



- **Example:**
  - Simplest approach: fit something **linear**, like $g(x_i; w) = g(x_i; a, b) = a + b \cdot x_i$
  - …where *a* and *b* are model parameters to be fitted!

Graphs adapted from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

Optimising parameters for a linear approach

Example:

— Simplest approach: fit something **linear**, like

$$g(x_i; w) = g(x_i; a, b) = a + b \cdot x_i$$

Linear regression

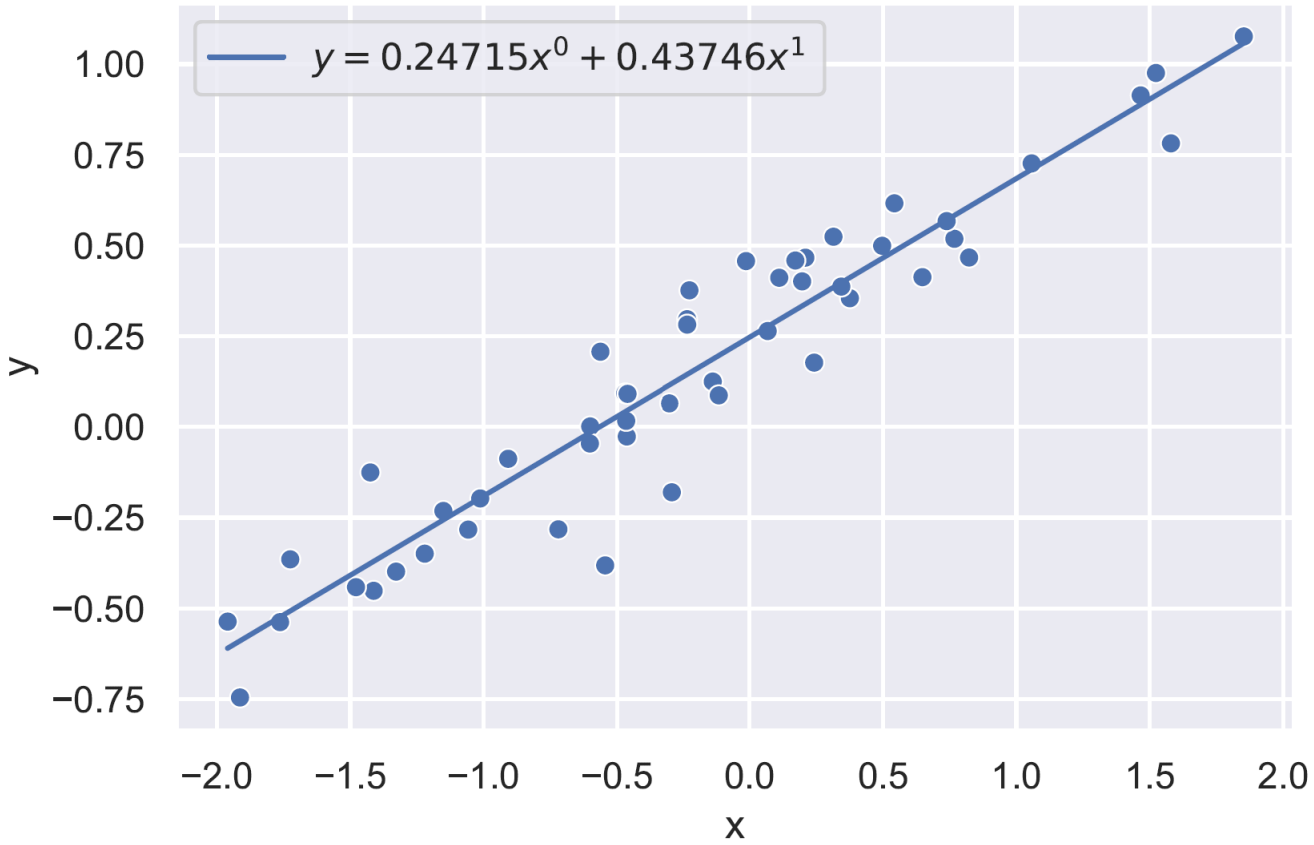
— will fit parameters a and b to minimise loss, i.e. MSE:

$$L_{\mathrm{MSE}}\big(y, g(x; w)\big) = \frac{1}{n}\sum_{i=1}^{n}(y_i - g(x_i; w))^2$$

$$L(w) = \|Xw - y\|^2$$

— can be done **analytically**, i.e. directly from the dataset. Closed-form solution:

$$w = (X^T X)^{-1} X^T y$$



Graphs adapted from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

Linear models for polynomials…

Linear regression:

— can be applied to any polynomial (which is still **linear in coefficients!**):

$$g(x_i; w) = a \cdot x_i^0 + b \cdot x_i^1 + c \cdot x_i^2 + \ldots$$

— Different models based on loss functions (avoid overfitting):

  — **Ridge** (L2): $L(w) = MSE + \lambda \|w\|^2$

  — **Lasso** (L1): $L(w) = MSE + \lambda \|w\|_1$

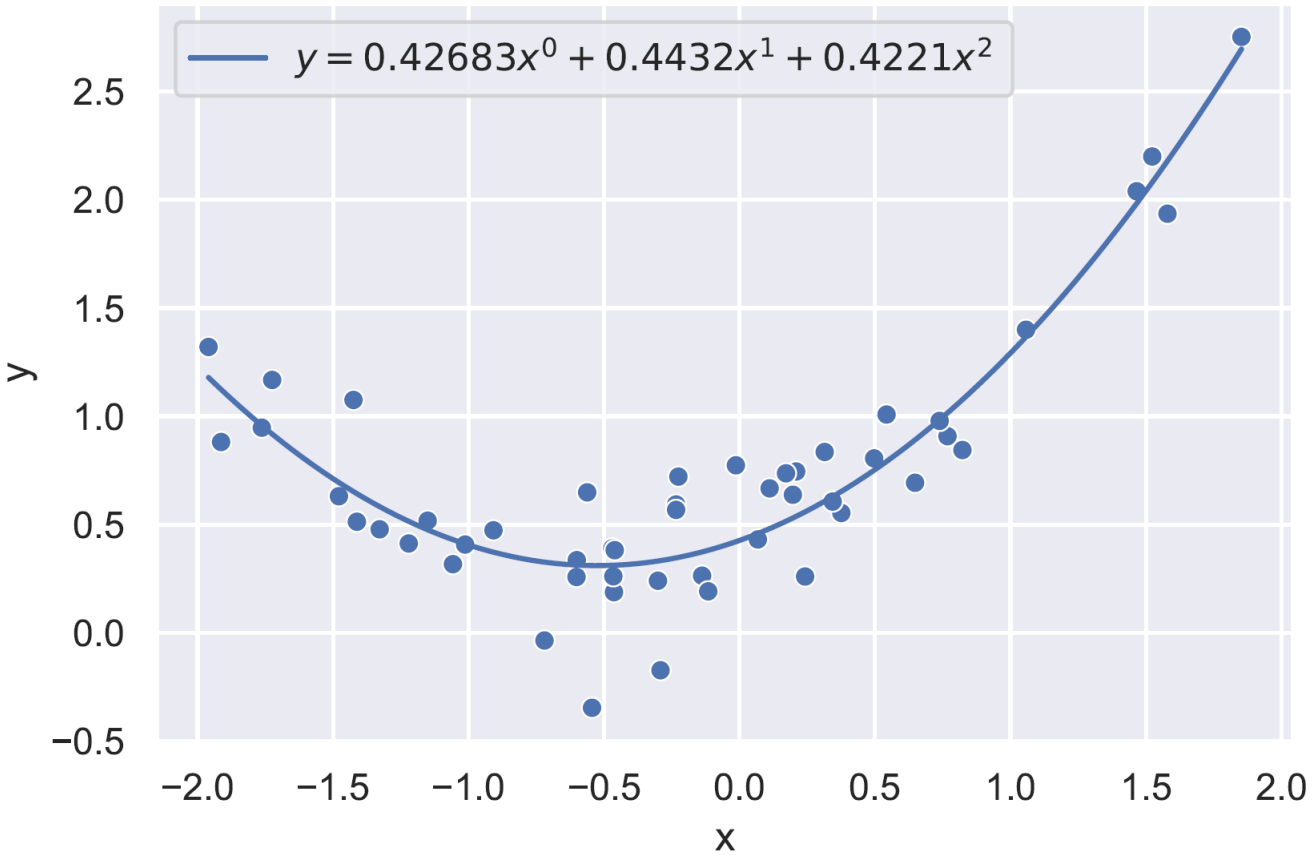
  — **ElasticNet**: Combining L1 and L2 in certain ratio

— Effect:

  — penalise large weights / coefficients

  — control model complexity

  — improve generalisation



— 1D feature: line

— 2D features: plane

— *n* features: hyperplane!

Graphs adapted from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

Comparison of different regression models

Linear and non-linear regression models are to be compared on the diabetes dataset.

Evaluate the model performance and discuss the metrics for evaluation.

— Which method gave the lowest prediction error?

— Which method produced the sparsest model?

Think of a larger dataset, e.g. gene expression:

— Why might Elastic Net work better than Lasso when genes are correlated?

— In a genomics experiment with 20000 genes (samples: maybe 50-100 patients), which method would you choose?

Linear Regression: Problems

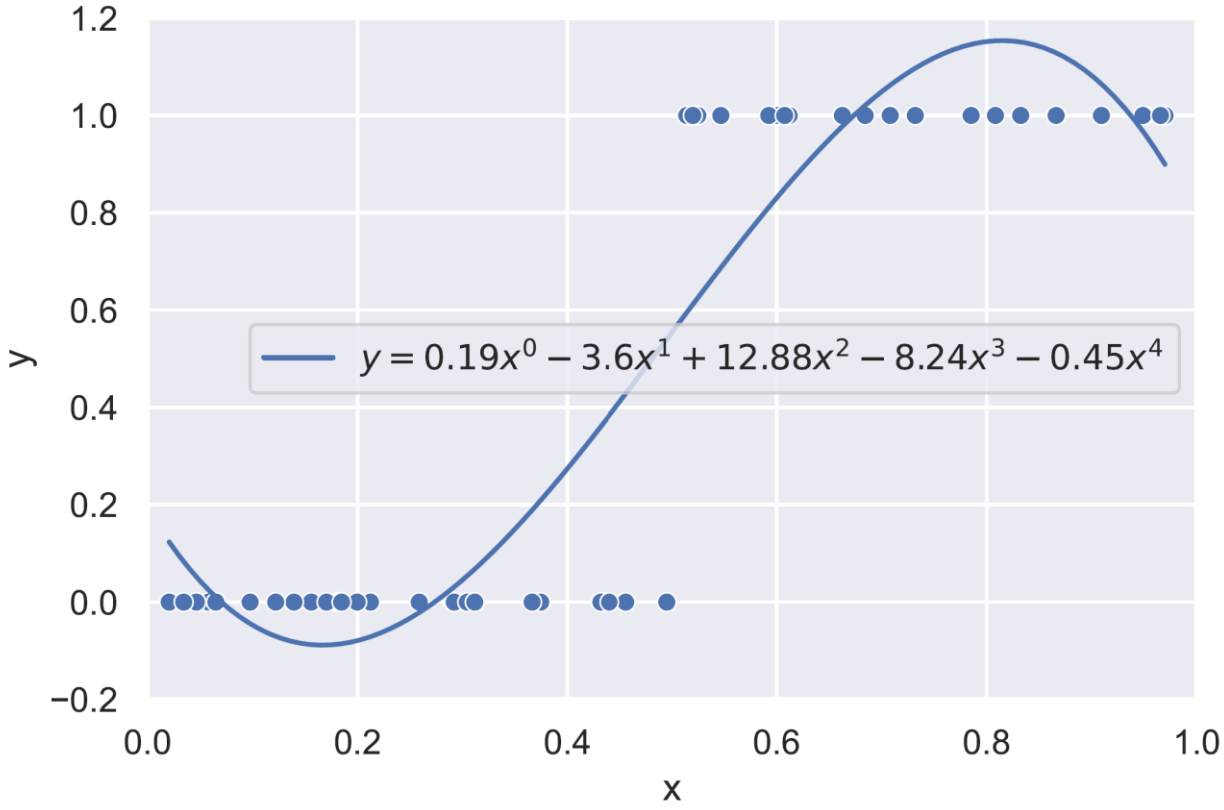
Imagine: **Classification task**

— First (bad) idea: fit a linear regression line

— Then, get classes based on some threshold:

$$y_i = \begin{cases} 0 \; g_{\text{LinReg}}(x_i; w) \leq 0.5 \\ 1 \; g_{\text{LinReg}}(x_i; w) \geq 0.5 \end{cases}$$

— Problem: Relationship between *X* and *y* simply **not linear**!



Graphs adapted from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

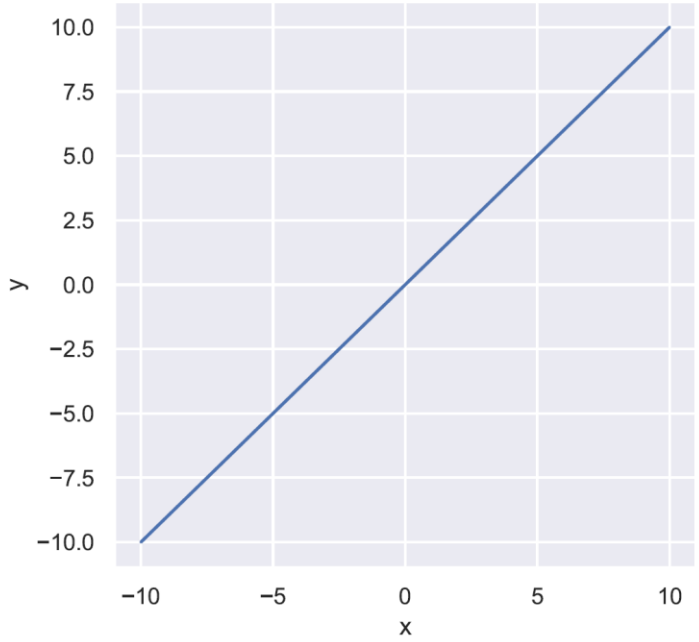
Logistic regression

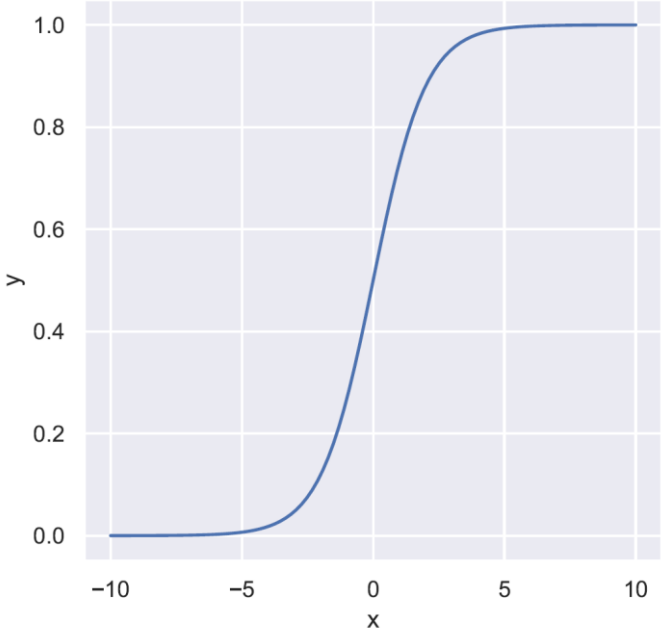
Instead: use non-linear function, e.g.

— **Logistic function (sigmoid function)!** $\sigma(z) = \dfrac{1}{1 + e^{-z}}$

— …where *z* is a linear function of features

— Logistic model = “wrapped” linear model!





Graphs adapted from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

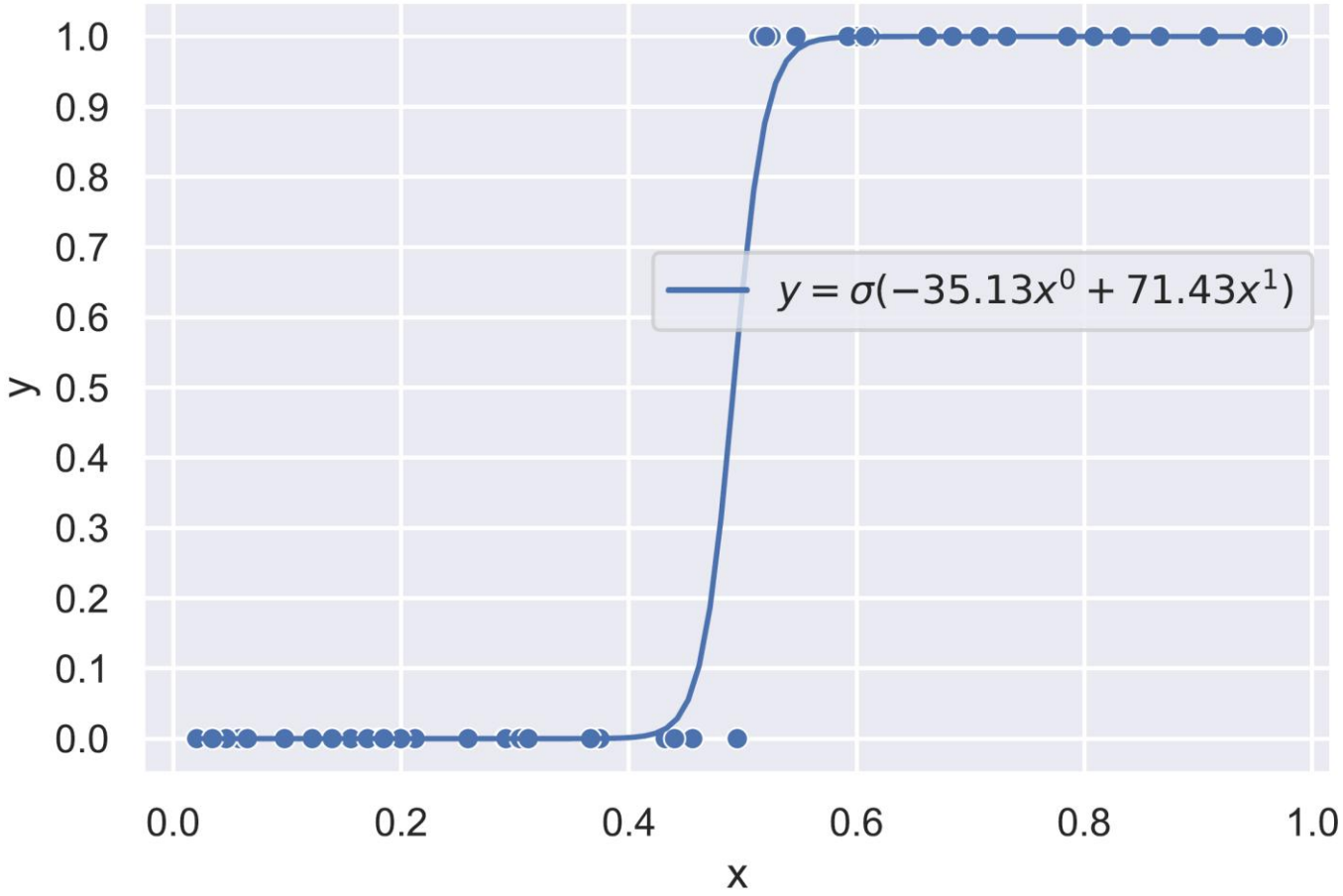
Logistic Regression

Imagine: **Classification task**

— Better idea: fit a logistic regression line

— Then, get classes based on some threshold:

$$y_i = \begin{cases} 0 \; g_{\text{LogReg}}(x_i; w) \leq 0.5 \\ 1 \; g_{\text{LogReg}}(x_i; w) \geq 0.5 \end{cases}$$



— Question: Regression or Classification?

— Problem with logistic regression:

  — **No closed-form solution**: minimum loss cannot calculated from data (no analytical solution)!

  — For minimising loss, **iterative methods** needed, e.g. …

Graphs adapted from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

Gradient descent

Given a function $f(x)$: Find x so that $f(x)$ is minimised

Idea:

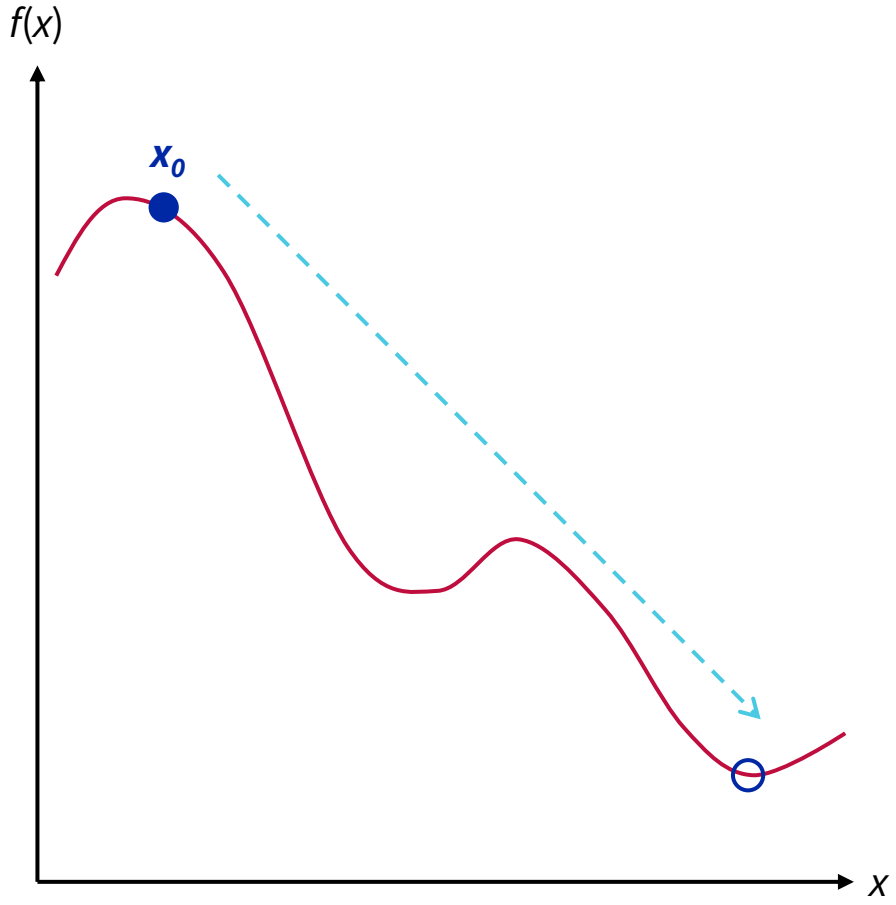
— start at some value x_0 …

— take a small step η into the direction where $f(x)$ decreases the strongest

Loss function minimised **iteratively**:

$$\theta_{n+1} = \theta_n - \eta \cdot \frac{\partial L(\theta_n)}{\partial \theta_n} = \theta_n - \eta \cdot \nabla L(\theta_n)$$

— Where η is the learning rate and θ is the (set of) parameter(s) to be optimised



From Logistic Regression to DL…

- Important concepts:
  - **Weights** (model parameters)
  - **Bias** (= intercept)
  - **Loss** functions
  - **Gradient descent**
  - Nonlinear activation (e.g. **sigmoid**)

Model weights determined via gradient descent

activation function

bias

$$p(y = 1 \mid x) = \sigma(\underbrace{w^{\mathrm{T}} x + b}_{\text{neuron preactivation (linear combination)}})$$

neuron output

  - Logistic regression = **neural network of one single neuron**.

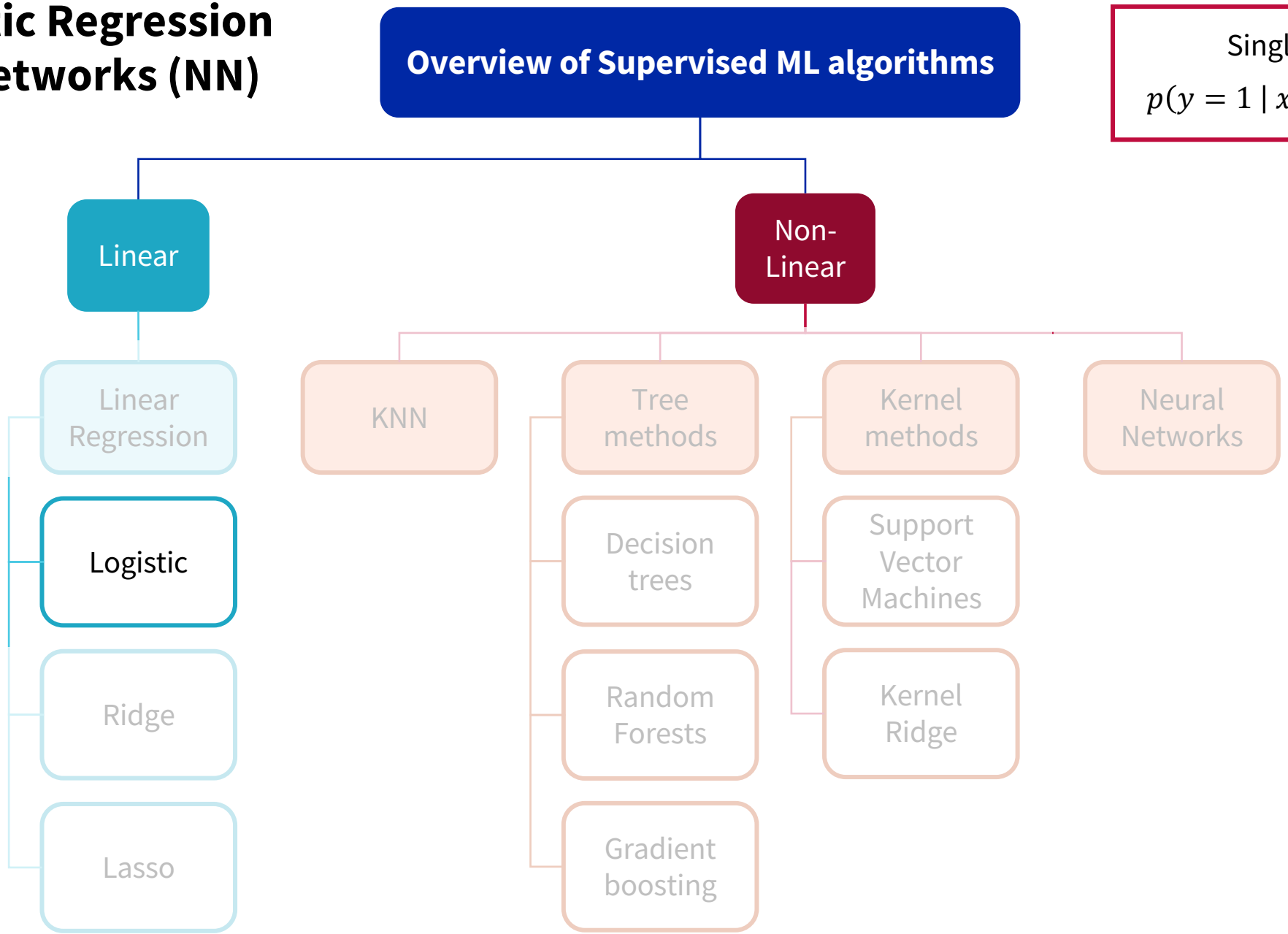
Logistic Regression

- No hidden layers
- Only one neuron
- Optimises log-loss using convex optimisation
- Used for binary classification

Neural Network Neuron

- Same mathematical unit
- But can be part of many layers
- non-convex optimisation
- different activations (ReLU, tanh, etc.)

From Logistic Regression to Neural Networks (NN)



Single neuron:

$$p(y = 1 \mid x) = \sigma(w^{\mathrm{T}} x + b)$$

A first glimpse on Neural Networks

Neurons / Perceptrons:

— Basic computational unit of a neural network

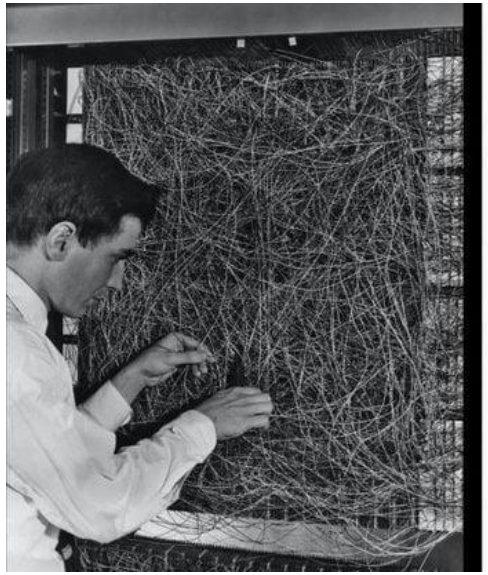
— Takes a vector of inputs, computes a weighted sum plus bias ($z = w^{\mathrm{T}}x + b$)

— Applies activation function ($a = \Phi(z)$)

— If activation potential passed: Output

— Weights and bias learned from data during training

— Concept developed 1957 by Frank Rosenblatt:



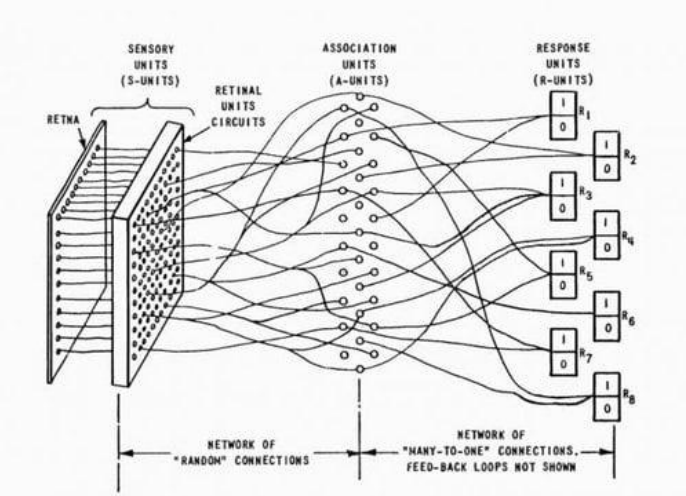
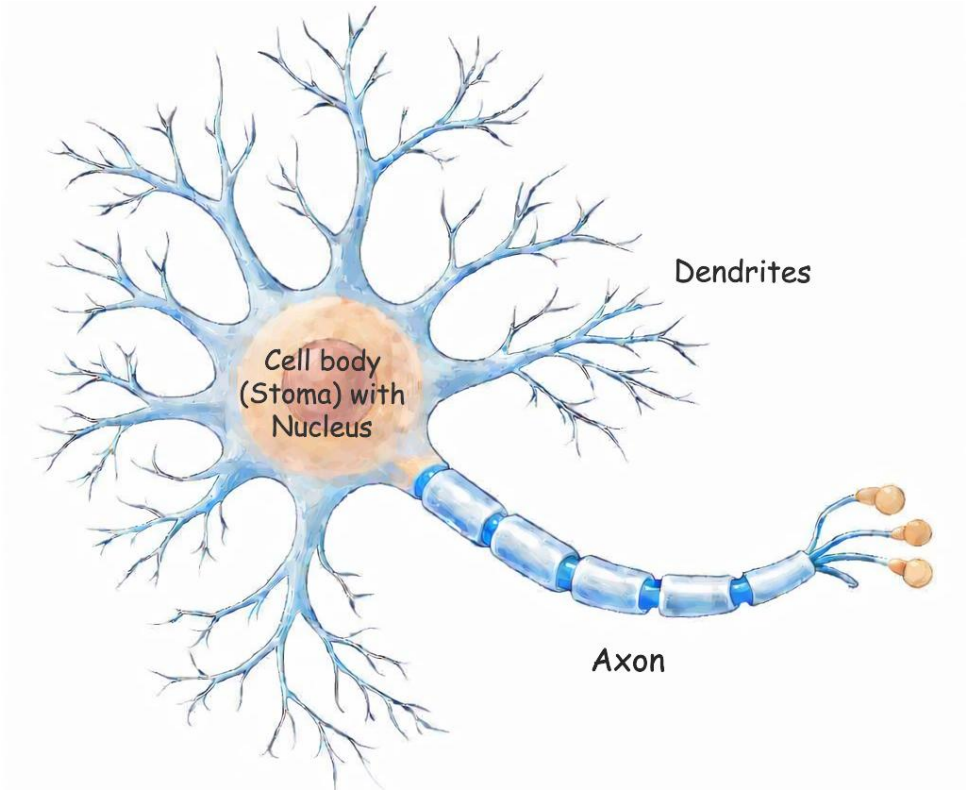
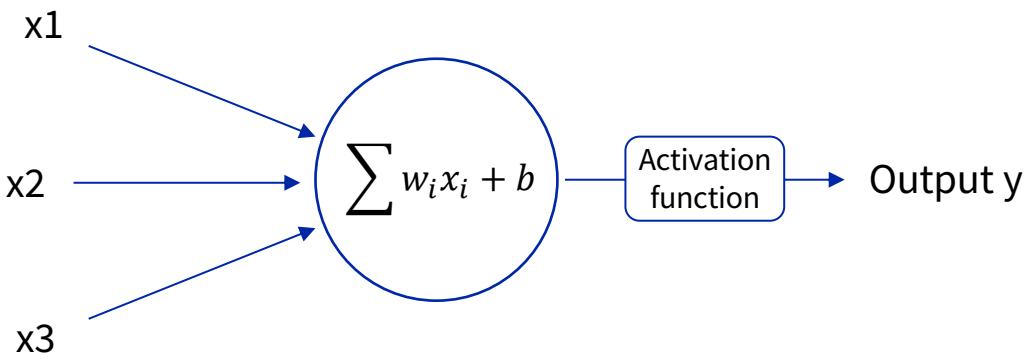


Figure 1 ORGANIZATION OF THE MARK I PERCEPTRON





F. Rosenblatt, *Psychol. Rev.* **1958**, *65*, 386-408, e.g. available in: J.A. Anderson, E. Rosenfeld, "Neurocomputing, Volume 1: Foundations of Research", MIT Press, **1988**; https://doi.org/10.7551/mitpress/4943.003.0010.
Photo from: https://data-science-blog.com/blog/2020/07/16/a-brief-historyof-neural-nets-everything-you-should-know-before-learning-lstm/

A first glimpse on Neural Networks

Layer:

— A group of neurons operating in parallel: $z = Wx + b$

— Input layer

— Hidden layer(s)

— Output layer

Epochs:

— Training iterations

Forward pass:

— Computing predictions through the network

Backpropagation:

— Efficient computation of gradients via the chain rule

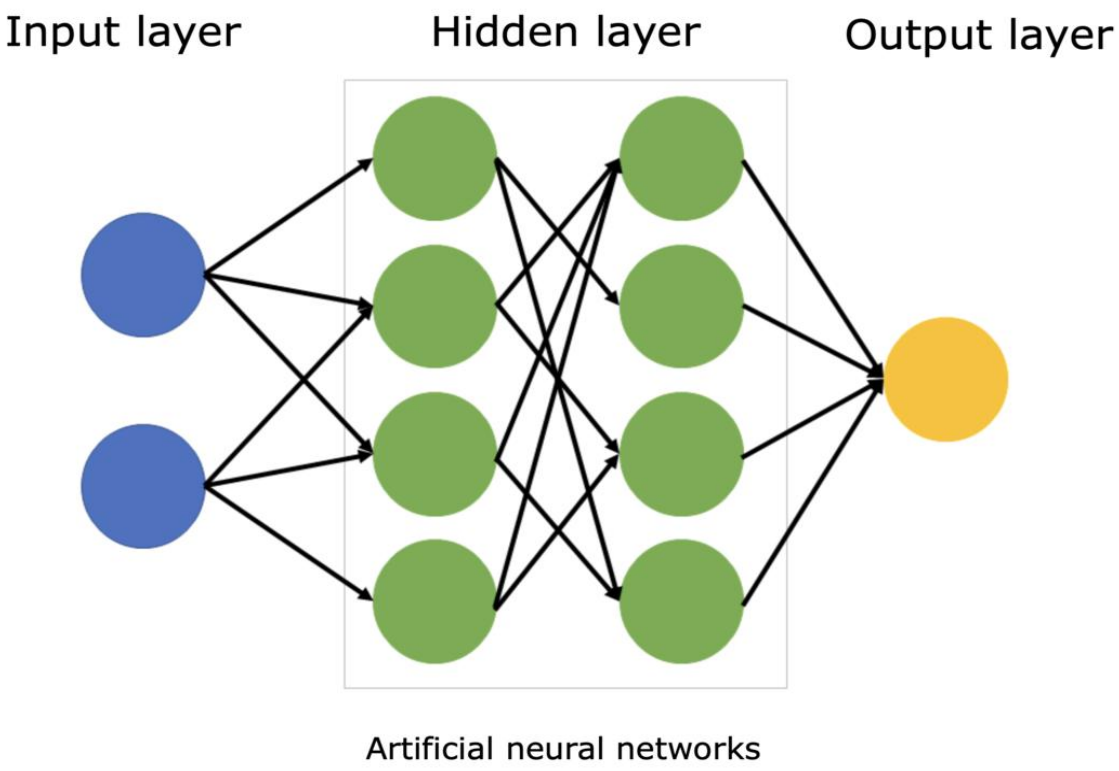


Image by seKiChun, CC BY-SA 4.0, https://www.sciencelearn.org.nz/images/5156-neural-network-diagram

Introducing PyTorch

- **Open-source deep learning platform**
- Well-suited for experiments and prototyping (also production):
  - Easy to build **computational graphs**
  - Automated computation of gradients (**autograd**)
  - Operates with CPU and GPU
  - High execution efficiency

- Three levels of abstraction:
  - **Tensors** represent tensors of any order (e.g., a scalar is a 0th order tensor), technically equivalent to numpy arrays
  - **Computational graphs**: PyTorch tensor = node in computational graph
  - **Modules**: layers for neural networks, allow for composition.

ML Concept	PyTorch
weights	`nn.Linear`
activation	`torch.relu`, `torch.sigmoid`
model	`nn.Module`
forward pass	`forward()`
training	optimizer + loss

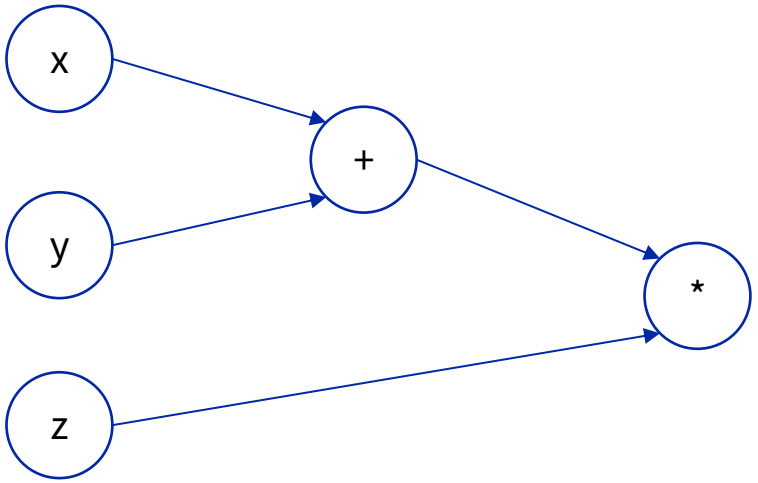
https://pytorch.org/

Computational graphs

A computational graph is a way to represent a math function in the language of graph theory:

- **Graph theory**:
  - Nodes connected by edges
  - Everything in the graph is either a node or an edge
- **Nodes:**
  - Input values
  - Functions for combining values
- **Edges** receive their weights as the data flows through the graph

E.g. a simple expression: $f(x, y, z) = (x + y) \cdot z$



https://pytorch.org/

Introduction to PyTorch

Run and inspect the minimal example for a NN in Pytorch:

— See how tensors work

— How NN are defined

— How they are trained and evaluated

Outlook: Neural Networks

— perceptron model

— activation functions (ReLU, tanh)

— multilayer networks

— backpropagation

— representation learning

— …

Deep learning is built from simple building blocks (**Linear models → logistic regression → neural networks**)